

复现代码与结果说明

《第三周周报》配套材料 · 实现要点与实测数据

实习项目: MotrixLab 线上实习 (第一月) 本周课次: 第 3 周 代码位置: `code/` (8 个脚本 + 4 个共享模块)

一、概述

本文是《第三周周报》的配套材料。周报只讲理论 (概念、推导与判据), 本文负责说明三件事: **写了哪些代码、每个文件的实现要点、以及跑出来的实际结果。**

所有脚本都遵循同一套约定, 以便 “结论可核对、过程可复现”:

- 每个脚本都可以**独立运行**, 内置断言全部通过后会打印 `all assertions passed`; 断言失败即退出码非零, 可直接用于 CI。
- 关键数学都配了**独立的交叉验证源**, 避免 “用同一个错误实现验证自己” 的假阴性: 正运动学对解析闭式解、解析雅可比对有限差分、SLERP 对解析转角、GAE 对 $\lambda = 0$ 与 $\lambda = 1$ 两个退化端点。
- 图落盘到 `figures/`, 数值结果落盘到 `results/`; 强化学习部分的脚本还会把指标写成 JSON, 供汇总脚本读取。
- 随机性全部显式播种 (`torch`、`numpy` 与环境重置), 因此同一脚本重复运行结果完全一致。

依赖: `numpy`、`scipy`、`matplotlib`、`gymnasium`、`torch` (CPU 即可, 无需 GPU); 报告生成部分额外需要 `tectonic` (LaTeX 引擎)。

二、代码模块说明

2.1 共享模块

模块	职责	被谁使用
se3.py	SE(3)/SO(3) 基础工具：旋转矩阵、欧拉角、四元数、旋转向量互转，万向锁处理，SLERP，齐次矩阵构造/求逆/作用	01、02、03
traj.py	梯形速度剖面（LSPB）：profile 生成、逐时刻求值、固定时长下的最优加速时间搜索	03、04
rl_common.py	CartPole 环境封装、状态离散化、统一评估协议、随机基线、结果 JSON 落盘	05、06、07、08
mpl_style.py	matplotlib 中文字体探测与设置（含无中文字体时的降级提示）	全部绘图脚本

表 1：四个共享模块的职责划分。

之所以把这些抽出来，是因为它们被多处复用且容易写错：**se3.py** 里所有函数都在 docstring 里写明了约定（四元数一律 xyzw、欧拉角一律 ZYX、 T_{ij} 表示 j 系在 i 系中的位姿），目标是“约定只在一处定义，中间层永不混用”；**traj.py** 被 03 和 04 共用，避免同一套速度剖面逻辑写两遍而产生不一致。

2.2 位姿变换（01_pose_transform.py）

覆盖六个方面：旋转矩阵基本性质、欧拉角与旋转矩阵往返、旋转矩阵与四元数往返、万向锁、齐次矩阵链式法则、SLERP 匀速性。核心设计是把“底层数学”全部放在 se3.py，本脚本只负责**验证约定 + 出图**，因此断言写得很直接。

两处值得说明的实现选择：

- **万向锁用显式分支处理**。当 $|\sin(\text{pitch})|$ 接近 1 时，把 roll 置零、把等效转角全部并入 yaw，并断言“重构出的姿态与原姿态严格等价”。这样即使用户传入奇异附近的欧拉角，也不会产生 NaN 或跳变。
- **解析求逆与数值求逆对照**。脚本同时用闭式解 $T^{-1} = [[R^T, -R^T p], [0, 1]]$ 和 `np.linalg.inv` 求逆并断言两者一致，既验证了公式，也说明生产代码里应当用前者。

2.3 正运动学与逆解（02_forward_kinematics.py）

用 UR5 的标准 DH 参数搭 6 轴链，验证四件事：DH 变换本身、链式累乘、几何雅可比、数值逆运动学。

雅可比用几何法构造（线速度列 $J_v = z_i \times (p_{\text{end}} - p_i)$ ，角速度列 $J_w = z_i$ ），再用有限差分法独立算一遍作对照。姿态部分的差分用旋转矩阵的反对称部分提取旋转向量，容差放宽到 $1\text{e-}5$ （因为依赖差分步长）。

逆解用阻尼最小二乘（Levenberg–Marquardt 风格）迭代：

$$\Delta q = J^\top (JJ^\top + \lambda^2 I)^{-1} e \quad (1)$$

阻尼项 λ 的作用是在奇异位形附近保持解有界；脚本另外做了单步限幅（避免奇异附近跳出一大步）和关节限位裁剪。为了说明阻尼的必要性，脚本专门构造了一个腕部奇异位形（ $q_5 = 0$ ，关节 4 与关节 6 轴线共线），断言此时最小奇异值趋于零，并对比裸伪逆与阻尼解的关节速度范数。

2.4 轨迹规划 (03_trajectory_planning.py)

实现了三类剖面与一个判据函数：

- **三次多项式**：闭式解系数（ $a_0 = q_0$ 、 $a_1 = 0$ 、 $a_2 = 3\Delta q/T^2$ 、 $a_3 = -2\Delta q/T^3$ ），两端速度为零。
- **五次多项式**：六个边界条件（两端的位置、速度、加速度）解 6×6 线性方程组。
- **LSPB 梯形剖面**：由 traj.py 提供，自动判断是梯形还是三角形。
- **plan_with_fixed_duration**：给定总时长，解析地求最优加速时间（ $t_a^* = \min(v_{\text{max}}/a_{\text{max}}, T/2)$ ），并给出可行性结论。

多项式求值不使用数值差分，而是对系数逐阶求导得到速度与加速度，这样峰值是精确的、不受差分步长影响。这一细节很重要：如果用差分去检查峰值是否超限，差分误差会让边界情况误判。

脚本还实现了笛卡尔直线 + 姿态 SLERP，并用“关节空间线性插值 + 正运动学”造出对比弧线，用来量化“关节空间插值末端不走直线”这个结论。

2.5 采样法路径规划 (04_rrt_planner.py)

在 $1\text{ m} \times 1\text{ m}$ 的平面里放了三个圆形障碍和一个矩形障碍，从 $(0.08, 0.08)$ 走到 $(0.92, 0.92)$ 。完整走一遍规划流水线：

碰撞检测 -> RRT / RRT* 采样 -> 捷径平滑 -> 按 $(v_{\text{max}}, a_{\text{max}})$ 时间参数化

碰撞检测用**线段密集采样**（步长 5 mm）而不是解析求交。这是一个有意的取舍：解析求交要多写几十行且各有边界情况，而采样步长比最小障碍尺寸（90 mm）小一个数量级，不会穿透。代码里用注释标明这个上限以及升级路径。

RRT* 相对 RRT 只多了两处：**choose parent**（邻域内选累计代价最小的父节点）与 **rewire**

(邻域内其他节点改认新节点，若这样更省)。时间参数化沿路径**弧长**套用 traj.py 的梯形剖面，先算累计弧长、再按弧长插值出位置与切向速度。

2.6 表格 Q-learning 与 DQN (05_qlearning_dqn.py)

表格 Q-learning: CartPole 的观测是 4 维连续量且部分无界，所以先裁剪到有限区间再分箱。Q 表用 4 维数组 $Q[x, \dot{x}, \theta, \dot{\theta}, a]$ 存储，正好可以用离散化返回的下标直接索引，避免手工做扁平化换算。

DQN: 实现了四个关键机制，每一个都对应 deadly triad 的一个病：

- **经验回放**——用预分配的 numpy 数组做环形缓冲，采样时随机取小批量，打散时间相关性。
- **目标网络**——每 200 步同步一次参数，稳住自举目标。
- **Double DQN**——在线网络选动作、目标网络估值，抑制 max 的高估。
- **Huber 损失**——防止时序差分误差的尖峰把梯度带飞。

另外脚本按固定间隔做贪心评估并**保留最优快照**，理由是 DQN 训练中途会崩（见 4.5 的实测曲线），只交最后一个网络是不可靠的。这也是业界默认模型选择做法。

2.7 策略梯度 (06_reinforce_actor_critic.py)

三种变体共用一个训练循环，**只换优势函数**，以保证对比公平（网络结构、随机种子、episode 预算完全相同）：

方法	优势函数	评论家是否训练
REINFORCE	$A_t = G_t$ （整条轨迹的折扣回报）	不训练（结构上保留但不用）
REINFORCE + 基线	$A_t = G_t - V(s_t)$	用蒙特卡洛回报拟合
A2C	$A_t = r_t + \gamma V(s_{t+1}) - V(s_t)$	用自举目标训练

表 2: 三种策略梯度变体的唯一差别。

两处工程要点：一是采样期间**不开梯度、不建计算图**，只记录 states/actions/rewards, episode 结束后对整条轨迹做一次批量前向算 log 概率与熵——逐帧建图会把 500 步的计算图全留着，速度差好几倍；二是 A2C 的自举**必须在真正终止处切断**（截断是时间上限、不是任务结束，仍要自举），脚本为此单独写了断言。

为了量化“基线降方差”这件事，脚本做了一个受控实验：**固定一个策略、采同一批 60 条轨迹，只换优势函数的形式**，比较信用分配信号的标准差。之所以要固定策略，是因为直接拿训练过程中的回报离散度去比会混进混杂因素（某个方法收敛差、episode 短，回报本来就集中），结论会被误导。

2.8 PPO 与裁剪消融 (07_ppo.py)

PPO 部分实现了：Actor-Critic 共享躯干网络、GAE(λ) 优势估计、重要性采样 + 裁剪目标、多 epoch 小批量更新、优势归一化、梯度裁剪、学习率线性退火。

脚本的核心产出是一次**严格消融**：其他所有超参完全相同，只把 ε 设为无穷大（等价于不裁剪），对照两者的近似 KL、裁剪生效比例与学习曲线。为了让消融能说明问题，这里**故意没有开启**“KL 超限提前结束 epoch”这道保险——否则它会先于裁剪生效，消融就看不出裁剪本身的作用了。

脚本还用四条断言把裁剪目标的性质逐项钉死：

- ratio 在 $[1-\varepsilon, 1+\varepsilon]$ 内时，梯度等于优势（裁剪不生效）；
- 正优势且 ratio 超过上界时，梯度为 0（策略不能再往上推）；
- 正优势且 ratio 低于下界时，梯度仍为优势（鼓励往回走）；
- 负优势情形对称：ratio 过小时梯度为 0；ratio $\equiv 1$ 时目标退化为普通策略梯度。

2.9 汇总与 A3C 实验 (08_algorithm_summary.py)

读取 05/06/07 的结果文件，汇总成横向对比表；另外实现了一个 A3C 并行采样的方差实验：**固定一个策略**，让每个“worker”跑一条轨迹，统计“平均 B 条轨迹的梯度估计”的标准差，再对数据做 log-log 幂律拟合，看指数是否落在理论值 0.5 附近。

这个实验对**可复现性特别敏感**：每条轨迹的初始状态都必须显式播种，否则每次跑出来的数字都不一样。这一条在 5.2 里作为踩坑记录单独列出。

三、运行方式与耗时

```
cd code
python 01_pose_transform.py          # 第 1 周：坐标系与位姿变换
python 02_forward_kinematics.py
python 03_trajectory_planning.py     # 第 2 周：轨迹规划
python 04_rrt_planner.py
python 05_qlearning_dqn.py          # 第 3 周：深度强化学习
python 06_reinforce_actor_critic.py
python 07_ppo.py                    # 依赖 06 的结果文件
python 08_algorithm_summary.py       # 依赖 05/06/07 的结果文件
```

注意：05–08 需要按顺序运行，因为 07 会读取 06 写下的 REINFORCE 曲线，08 会读取 05/06/07 三份结果文件。01–04 之间没有依赖，可以任意顺序运行。

脚本	对应内容	产出	实测耗时
01_pose_transform.py	齐次矩阵、欧拉角、四元数、万向锁、SLERP	图 1	1.4 s
02_forward_kinematics.py	6 轴 DH 正运动学、雅可比、阻尼最小二乘 IK	图 2	1.2 s
03_trajectory_planning.py	三次 / 五次多项式、LSPB、笛卡尔直线 + SLERP	图 3、trajectory.csv	1.2 s
04_rrt_planner.py	RRT / RRT*、捷径平滑、时间参数化	图 4、rrt_path.csv	2.5 s
05_qlearning_dqn.py	表格 Q-learning vs DQN	图 5、rl_qlearning_dqn.json	约 2.5 min
06_reinforce_actor_critic.py	REINFORCE / + 基线 / A2C	图 6、rl_policy_gradient.json	约 4 min
07_ppo.py	PPO + GAE + 裁剪消融	图 7、rl_ppo.json	约 4.8 min
08_algorithm_summary.py	横向汇总 + A3C 并行方差实验	图 8、rl_summary.json	20 s

表 3：脚本清单与实测耗时（Windows / CPU 单机，总耗时约 12 分钟）。01–04 为确定性数值验证，秒级完成；05–08 是训练类任务，耗时集中在环境交互。

首次运行前若缺少依赖，可用：

```
pip install numpy scipy matplotlib gymnasium torch
```

报告与文档本身由 `__tools/build_pdf.py` 用 LaTeX (tectonic) 编译生成。它会把 content_

theory.py 与 content_code.py 两份内容文件转成.tex，再分别产出《第三周周报》与《复现代码与结果说明》；图中的实验数据同样是构建时从 **results/*.json** 读取的，所以重跑实验后再生成，数据会自动同步，不会出现“文档写的和代码跑的不一致”。

四、实验结果

4.1 位姿变换

脚本 01 的 6 组断言全部通过。图 1 的三个子图分别对应：两连杆臂的右乘链式 ((a))、万向锁在 $\text{pitch} \rightarrow 90^\circ$ 时的数值崩溃 ((b))、以及 SLERP 相对欧拉角线性插值的匀速性差异 ((c))。

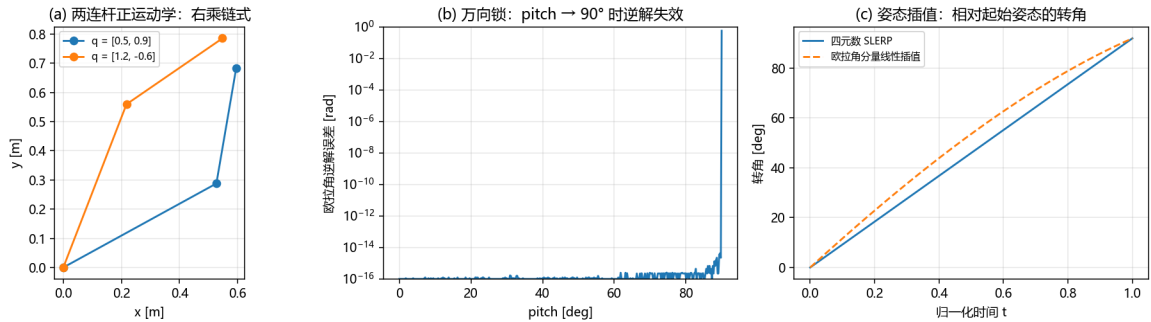


图 1：位姿变换的三个关键现象。(b) 中 pitch 逼近 90° 时欧拉角逆解误差从 $1e-16$ 量级跳到 $1e-1$ 量级，这就是万向锁；(c) 中 SLERP 的转角随时间是严格直线，而欧拉角分量线性插值不是匀速转动。

验证项	方法	实测结果
旋转矩阵性质	正交性 + 行列式 = +1	误差 $< 1e-12$ ，通过
欧拉角 $\leftrightarrow$ 旋转矩阵	ZYX 约定往返	误差 $< 1e-10$ ，通过
旋转矩阵 $\leftrightarrow$ 四元数	xyzw 约定往返	误差 $< 1e-10$ ，通过
万向锁处理	pitch = $\pm 90^\circ$ 时置 roll = 0	重构姿态严格等价，通过
解析求逆 vs 数值求逆	闭式解对照 np.linalg.inv	误差 $< 1e-12$ ，通过
链式右乘	$T_{03} = T_{01}T_{12}T_{23}$ 与逐步变换对照	误差 $< 1e-12$ ，通过
两连杆正运动学	齐次矩阵链 vs 解析闭式解	误差 $< 1e-12$ ，通过
SLERP 匀速性	等间隔采样的相邻转角	相等，误差 $< 1e-9$ ，通过

表 4：位姿变换模块的验证项与实测误差（脚本 01）。

一个具体算例：B 系相对 A 系绕 z 轴转 45° 、原点位于 (0.5, 0, 0.2)。把 B 系下的点 (1, 0, 0) 换算到 A 系，程序输出 (1.2071, 0.7071, 0.2)；手算 $(\cos 45^\circ, \sin 45^\circ, 0) + (0.5, 0, 0.2)$ 得到同样结果，数值与解析两侧互相印证。

4.2 正运动学、雅可比与逆解

取关节角 $q = (0.3, -0.8, 1.1, -0.5, 1.4, 0.2)$ rad，程序算出的末端位姿为 $[x, y, z, \text{roll}, \text{pitch}, \text{yaw}] = (-0.6984, -0.3449, 0.2015, 1.3711, -0.1623, -1.0639)$ ，四元数 $(0.512, -0.374, -0.347, 0.691)$ ；该位形下雅可比奇异值为 $(1.9176, 1.5522, 0.9852, 0.4585, 0.4450, 0.1688)$ ，可操作度 0.1010。

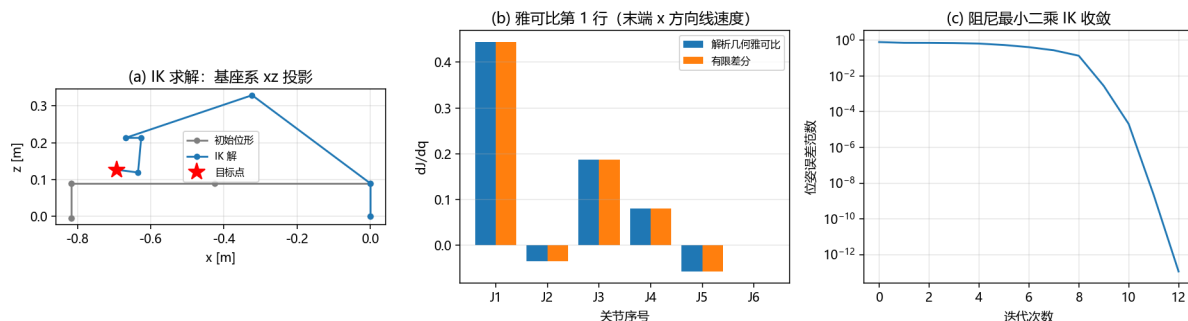


图 2：6 轴臂的正运动学与逆解。(a) 基座系 xz 投影下初始位形与 IK 解的对比，红星为目标点；(b) 解析几何雅可比与有限差分雅可比逐项吻合；(c) 阻尼最小二乘 IK 的误差单调下降，约 12 次迭代收敛到 $1e-12$ 以下。

检查项	做法	实测结果
DH 变换正确性	与教科书展开式逐项对照	误差 $< 1e-12$
链式累乘	不借助 dh_transform，按 $RzTzTxRx$ 顺序手写相乘	逐级一致，误差 $< 1e-12$
解析雅可比	几何法 vs 有限差分	位置误差 $< 1e-6$ ，姿态 $< 1e-5$
雅可比速度映射	$J \cdot dq$ 与末端位姿数值微分对照	误差 $< 1e-5$
数值逆运动学	阻尼最小二乘，容差 $1e-10$	约 12 次迭代收敛，位姿误差 $< 1e-7$
奇异位形	腕部对齐 ($q_5 = 0$) 时最小奇异值	$< 1e-6$ ，并按预期秩亏
奇异处阻尼必要性	对比裸伪逆与阻尼解	阻尼解的关节速度范数显著更小

表 5：正运动学 / 雅可比 / 逆解的验证结果（脚本 02）。

奇异位形这一项值得展开说：在 $q_5 = 0$ 时关节 4 与关节 6 的轴线共线，雅可比秩亏，裸伪逆会解出范数极大的关节速度，真机上会直接触发关节限速乃至撞机。脚本用阻尼形式 $dq = J^T(JJ^T + \lambda^2 I)^{-1}e$ ，并断言阻尼解的速度范数严格小于裸伪逆——这就是工业控制器里必开阻尼的原因。

4.3 轨迹规划

脚本 03 的实测结果验证了周报 3.1 与 3.3 的推导：对 $\Delta q = 1$ rad、 $T = 1$ s 的三次多项式，程序算出的峰值速度与峰值加速度为 1.500 rad/s 与 6.000 rad/s²，与闭式解 $1.5\Delta q/T$ 、 $6\Delta q/T^2$

完全吻合。而设定的上限是 $v_{\max} = 0.6 \text{ rad/s}$ 、 $a_{\max} = 2.0 \text{ rad/s}^2$ ，也就是说这条曲线**两项都超限**（分别超 2.5 倍与 3 倍）——这正是“必须显式限制速度加速度”的算术依据。

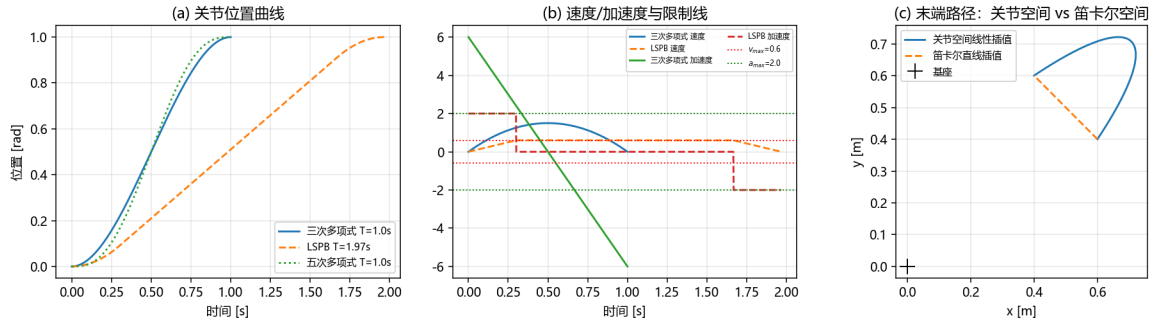


图 3: 轨迹规划。(a) 三次 / 五次多项式与 LSPB 的位置曲线；(b) 速度、加速度与限制线——三次多项式的两条曲线都冲出了虚线限制，LSPB 恰好压在线上；(c) 关节空间插值走弧线，笛卡尔直线插值走直线。

固定时长规划器的实测输出与表 1（周报）的算例一致：行程 0.8 rad 的自然时长为 1.633 s，最优峰值速度为 0.471 rad/s、峰值加速度 1.569 rad/s²，2 s 内可行；行程 1.2 rad 的自然时长 2.300 s，最优峰值 0.706 rad/s 与 2.353 rad/s²，两个限制同时超 18%，2 s 内不可行。脚本还断言了解析最优解不劣于 2001 点网格搜索，以及“可行 $\Leftrightarrow$ 自然时长 $\leq T$ ”这条等价判据。

笛卡尔与关节空间的对比也用数值确认了：笛卡尔直线插值的路径长度与两点直线距离完全相等 (0.707107 m，误差 $< 1e-9$)，而同样的起终点在关节空间线性插值时，末端走出的弧线比弦长多 1% 以上。

4.4 采样法路径规划

同一随机种子下：RRT 得到 90 个节点、路径长 1.4572 m；RRT* 同样 90 个节点、路径长 1.3191 m，缩短约 9.5%；再用随机捷径平滑压到 4 个途经点、1.2313 m，比 RRT 原始解短 15.5%。

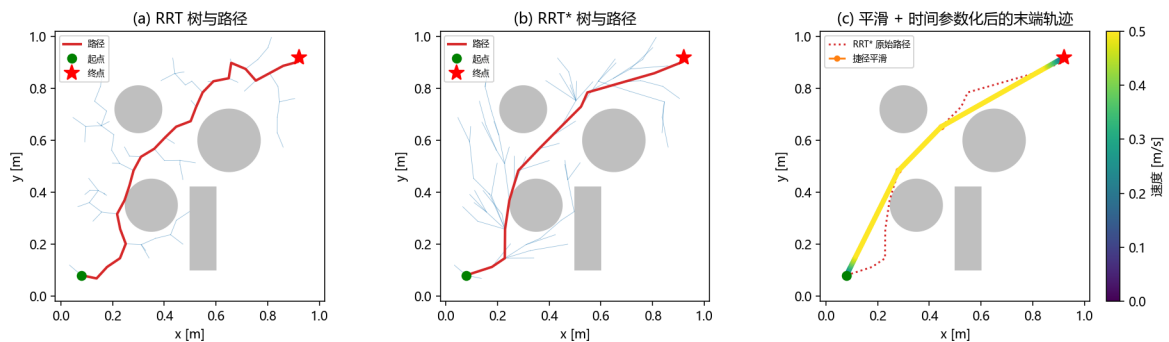


图 4: RRT 与 RRT* 规划结果。(a)(b) 采样树与最终路径，蓝色细线是搜索树，红线是路径；(c) 捷径平滑后再按弧长做时间参数化得到的末端轨迹，颜色表示瞬时速度，两端为加减速段，中段达到 $v_{\max}$ 。

时间参数化取 $v_{\max} = 0.5 \text{ m/s}$ 、 $a_{\max} = 1.5 \text{ m/s}^2$ ，得到总时长 2.796 s，峰值速度 0.500 m/s、峰值加速度 1.500 m/s²，两项都恰好压在限制上且不越界；脚本同时断言了整条时间轨迹上的

采样点都不在障碍物里。这就是“路径-速度解耦”的完整落地。

4.5 Q-learning 与 DQN

评价协议统一为：贪心策略跑 50 个 episode 取平均回报。同时给出随机策略与恒定动作作为参考。

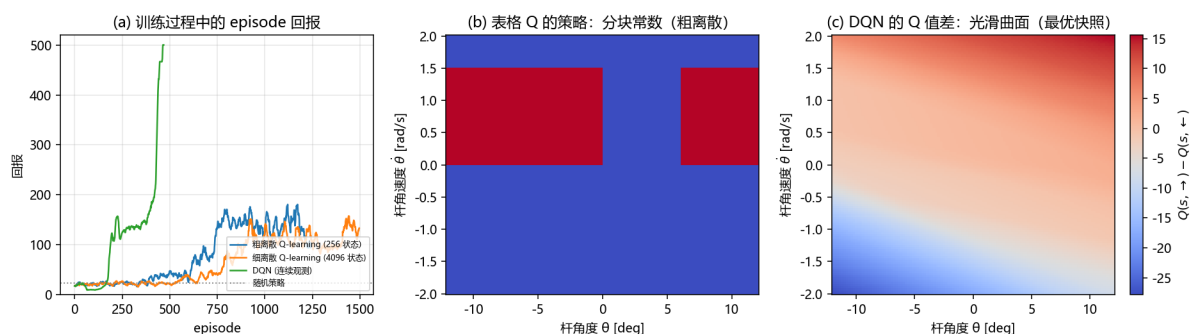


图 5: Q-learning 与 DQN。(a) 三种配置的训练曲线：两种表格法都卡在 150 分附近，DQN 打满 500；(b) 表格 Q 表学到的策略在 $\theta-\dot{\theta}$ 平面上是分块常数；(c) DQN 的 Q 值差（推右减推左）是一张光滑曲面，决策边界连续。

配置	状态数	贪心回报	说明
随机策略	—	23.1 ± 14.8	任务难度下界
恒定动作	—	9.3 ± 0.7	最差参考
表格 Q-learning	$4^4 = 256$	139.1 ± 67.3	离散化粗，状态覆盖好
表格 Q-learning	$8^4 = 4096$	152.7 ± 78.9	格子更细但更稀疏，提升有限
DQN（最后一个网络）	连续	500.0 ± 0.0	本随机种子下也达标
DQN（最优快照）	连续	500.0 ± 0.0	周期贪心评估选出的最好一版

表 6: 值方法的实测结果（脚本 05）。表格法的天花板是离散化误差，不是调参问题。

DQN 的训练过程本身也很能说明问题。脚本每 5000 步做一次贪心评估，实测序列为 5k:184 → 10k:99 → 15k:171 → 20k:125 → 25k:141 → 30k:180 → 35k:198 → 40k:473 → 45k:500 → 50k:500 → 55k:500（环境步：回报）。可以看到 10k 步时从 184 掉到 99 —— **训练到一半策略崩了**，之后才慢慢爬回来，40k 步后稳定在 500。这解释了为什么工程上一定要做周期评估 + 保留最优快照：只交最后一个网络是不可靠的。

4.6 三种策略梯度变体与方差实验

三种变体在相同的 1000 个 episode 预算下：

受控方差实验的结果：固定策略、同一批 60 条轨迹，只把优势函数从 G_t 换成 $G_t - V(s_t)$ ，信用分配信号的标准差从 **8.53 降到 5.74**，其中评论家解释了 54.7% 的回报方差。这直接验证

方法	贪心回报	平均 episode 长度	平均策略熵
REINFORCE	125.4 ± 7.5	371.1	0.533
REINFORCE + 基线	500.0 ± 0.0	397.6	0.533
A2C	500.0 ± 0.0	322.2	0.410
随机策略	23.1 ± 14.8	—	—

表 7：三种策略梯度变体的实测结果（脚本 06，1000 个 episode）。REINFORCE 在这个预算下没能解掉任务，加上基线后立刻达标——方差就是这个方法能不能工作的分水岭。

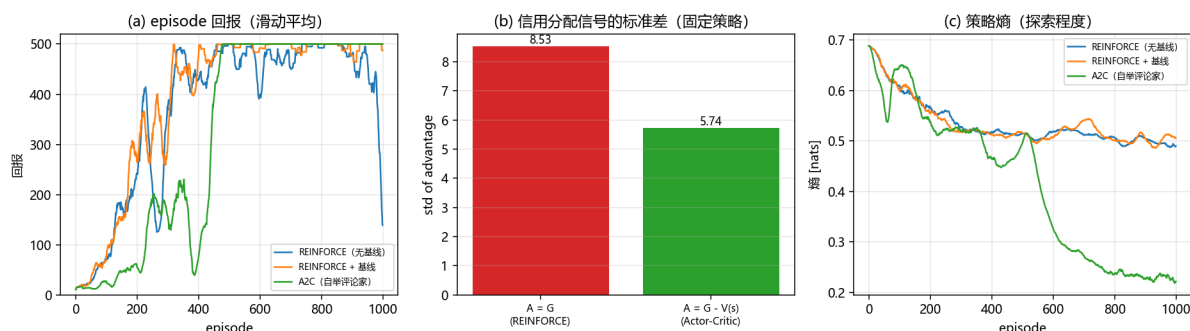


图 6：三种策略梯度变体。(a) 锯齿状的回报曲线是策略梯度方法方差大的直观表现——冲上去、崩下来、再学回来；(b) 固定策略下，减去基线把信用分配信号的标准差从 8.53 压到 5.74；(c) 策略熵的下降速度。

了周报 4.4 中“减基线不引入偏差、却能压低方差”的结论。

4.7 PPO 与裁剪消融

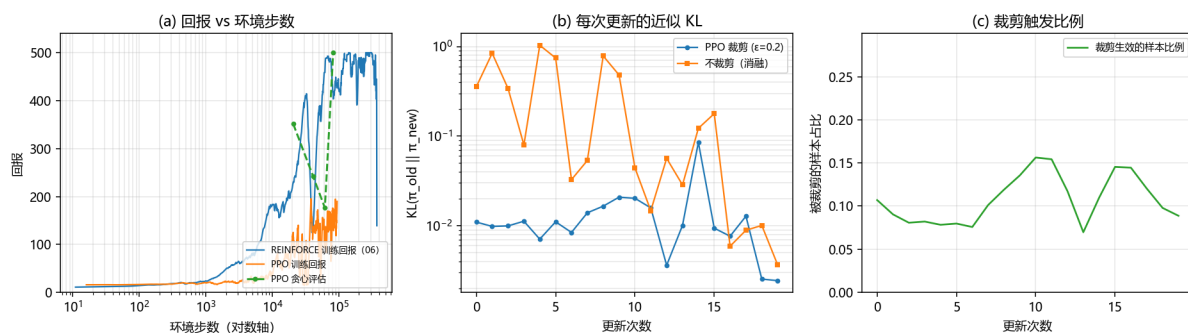


图 7：PPO 与消融。(a) 回报 vs 环境步数：橙线是 PPO 训练时随机策略的回报，绿虚线是周期贪心评估；(b) 每次更新的近似 KL，裁剪后全程贴在 $1e-2$ 附近，不裁剪则在 $1e-2 \sim 1e0$ 之间剧烈震荡；(c) 裁剪真正生效的样本比例，稳定在 10% 左右。

也就是说，“稳定”可以量化成一句：裁剪把每次更新的策略变化量（以近似 KL 度量）压小了约 18 倍，而且这不是靠调小学习率换来的。裁剪生效的样本比例稳定在 10.4% 左右，说明这项机制确实在持续工作，而不是形同虚设。

4.8 A3C 并行采样与横向汇总

指标	PPO ($\varepsilon = 0.2$)	不裁剪 (消融)	差距
平均每次更新的近似 KL	0.01449	0.26130	18.0 倍
最大 KL	0.08523	1.03419	12.1 倍
裁剪生效的样本比例	10.4%	0.0%	—
贪心回报	500.0	(未单独评估)	—

表 8: 裁剪消融实验。两组的学习率、批大小、epoch 数完全相同，唯一差别是裁剪是否开启；不裁剪时每次更新把策略推得远约 18 倍。

固定策略下，让每个 “worker” 跑一条轨迹，统计 “平均 B 条轨迹的梯度估计” 的标准差：

并行 worker 数 B	1	2	4	8	16	32
梯度估计标准差	0.3411	0.2509	0.1958	0.1464	0.0932	0.0749
相对单 worker	100.0%	73.5%	57.4%	42.9%	27.3%	22.0%

表 9: 并行采样对梯度方差的削减。32 个 worker 把标准差压到单 worker 的 22.0%（理论上限 $1/\sqrt{32} \approx 17.7\%$ ）。

对数据做 log-log 幂律拟合，得到的幂指数为 0.447，与理论值 0.5（即标准差按 $1/\sqrt{B}$ 下降、方差按 $1/B$ 下降）吻合得很好，定量验证了周报 4.6 中 A3C 的方差缩减结论。

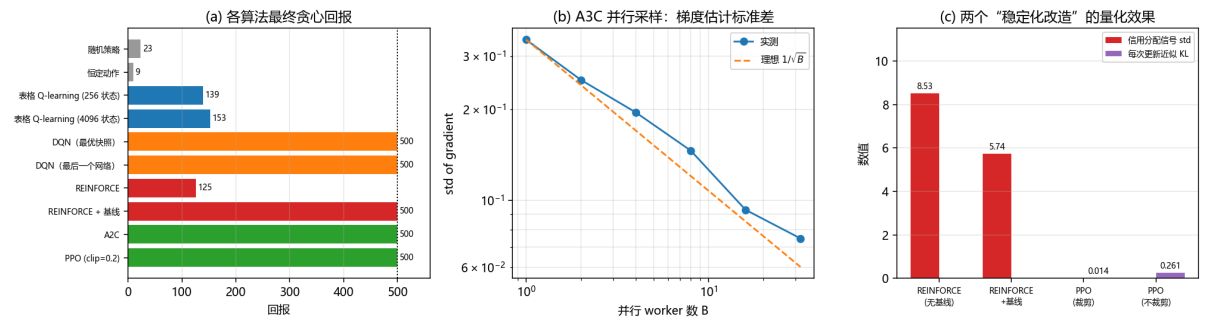


图 8: 横向汇总。(a) 各算法的最终贪心回报；(b) A3C 并行采样的方差缩减，实测幂律指数 0.447（理论 0.5）；(c) 两个 “稳定化改造” 的量化效果对比。

这张表还带出一个诚实的观察：在 CartPole 上，好几种方法最后都能打满 500 分，单看最终分数区分不出算法优劣。真正的差异在过程里——样本效率、梯度方差、训练曲线的抖动程度、以及对超参和随机种子的敏感度。所以本文才把 “信用分配信号的方差” 和 “每次更新的 KL” 这两个过程指标单独量了出来。

样本效率方面，PPO 用 10 万环境步达到满分，贪心评估在 81920 步首次摸到 500。作为

方法	类别	贪心回报	关键机制
随机策略	基线	23.1	CartPole 随机动作，任务难度下界
恒定动作	基线	9.3	始终推左，最差参考
表格 Q-learning (256 状态)	值迭代 / 表格	139.1	离散化粗，状态覆盖好
表格 Q-learning (4096 状态)	值迭代 / 表格	152.7	格子更细但更稀疏，提升有限
DQN (最优快照)	值函数逼近	500.0	连续观测 + 经验回放 + 目标网络
DQN (最后一个网络)	值函数逼近	500.0	同一配置的末态；训练中会崩，靠快照选择兜底
REINFORCE	策略梯度	125.4	无基线，梯度方差最大
REINFORCE + 基线	策略梯度	500.0	减去蒙特卡洛评论家 $V(s)$
A2C	演员-评论家	500.0	单步自举优势，偏差大方差小
PPO (clip=0.2)	演员-评论家	500.0	裁剪 + 同批数据复用 10 个 epoch

表 10: 十种方法与基线的横向对比（脚本 08 汇总 05/06/07 的结果文件）。

对照，REINFORCE 在同样的 1000 个 episode 预算下（约 371k 环境步，是 PPO 总预算的 3.7 倍）**根本没解掉任务**：贪心评估只有 125.4 分，而同一个训练循环一旦加上基线就变成 500 分。这个对照比单看分数更有意义：**REINFORCE 能不能工作，几乎完全取决于有没有一个好基线**，而 PPO 把这件事做成了算法的一部分。

还有一个值得记下的现象：REINFORCE 训练期间采样策略的平均回报能到 303 分，但把策略换成确定性的贪心动作后只剩 125.4 分。说明它的随机策略是“靠探索东一下西一下平均出来的”，而它的贪心路径本身是错的。这类“随机比贪心好”的策略在策略梯度里并不罕见，也是评估时必须固定用贪心、且要单独报告的原因。

五、问题与解决

5.1 实现层面踩过的坑

问题现象	根本原因	解决办法
两连杆正运动学与解析解对不上	<code>make_transform(R, p)</code> 构造的是“位姿” $[[R, p]]$ ，而连杆变换是 $\text{Rot}_z(\theta) \cdot \text{Trans}(l)$ ，两者的平移列不同；混用导致整条链错位	拆成 <code>rot_z_h</code> / <code>trans</code> 两个显式 helper，让复合顺序在代码里一眼可见
DH 链检查全部通过，但 FK 结果全错	链式检查用的是同一个错误的 <code>dh_transform</code> ，属于“自己验证自己”的假阴性；另外 DH 展开时漏掉 $R_z(\theta)$ 对平移列的作用	加一条独立检查，把 DH 矩阵与教科书展开式逐项对照
RRT 路径与障碍物重叠	线段碰撞检测的采样步长太粗，会从薄障碍中间穿过去	把采样步长降到 5 mm（远小于最小障碍尺寸），并留注释说明升级路径
表格 Q-learning 稳定卡在 150 分，加密格子反而更差	不是 bug，是离散化误差的天花板：格子越密，每个格子见到的样本越少	用 DQN（连续观测）做对照确认结论，并把这一现象写进结果说明
DQN 训练到一半策略崩溃（贪心评估 184 → 99）	每步都做梯度更新 + 学习率 $1e-3$ 过于激进，Q 值持续膨胀 ($ Q _{\max}$ 从 8 涨到 51)	学习率降到 $2.5e-4$ ，并按固定间隔做贪心评估、保留最优快照
REINFORCE 偶尔塌缩到 9 分（比随机还差）	学习率 $1e-2$ 时策略过早坍缩成确定性动作，探索彻底停止	学习率降到 $1e-3$ ，同时保留熵奖励项

表 11：实现层面遇到的主要问题与解决办法。

最值得记的一条是第二行那个“假阴性”：用同一个（错误的）实现去验证它自己，断言再多也没用。后来我给所有关键数学都配了独立的交叉验证源——正运动学对解析闭式解、解析雅可比对有限差分、SLERP 对解析转角、PPO 的 GAE 对 $\lambda = 0$ 与 $\lambda = 1$ 两个退化端点。

5.2 实验可复现性

训练类脚本的可复现性比数值脚本更容易出问题，中途踩了两个坑：

- **环境未播种。**只设 torch/numpy 的随机种子是不够的：episode 的初始状态由环境的随机数发生器决定，而 gymnasium 在创建环境时会用系统熵给它播种。结果就是“同样的种子，两次运行的指标不同”。解决办法是在训练开始时显式调用一次 `env.reset(seed=seed)`，后续的 `reset` 都从这条已播种的随机流继续推进——既保证初始状态多样，又保证整个训练过程可复现。
- **随机动作未播种。**更隐蔽的一个：`env.reset(seed=...)` 只保证初始状态可复现，而 `env.action_space.sample()` 用的是另一个随机源，不单独播种就每次都不一样。这会让“随机策略基线”这个参照值在每次运行时漂移，解决办法是显式调用 `env.action_space.seed(...)`。

两处修完后，重复运行 06 与 08 得到的 JSON 结果文件完全一致（逐字段比对通过），本说明中的所有数字都可复现。

另外还有一处与可复现性相关的设计取舍值得说明：DQN 脚本采用了“周期贪心评估 + 保留最优快照”。严格来说，这等于用评估集做了模型选择，所以报告中同时给出“最后一个网络”与“最优快照”两个数，两者之间的差距本身就是 DQN 训练不稳定的证据——而不是把它藏起来只报最好看的那个数。

六、结论

本套代码对周报中的每一条理论结论都给出了可执行的验证：

- **理论结论 → 验证方式：**位姿变换的约定 → 往返一致性断言；链式法则 → 与解析闭式解对照；雅可比 → 与有限差分对照；速度/加速度上限 → 与闭式峰值公式对照；“两秒可行性判据” → 与网格搜索最优解对照；Bellman 目标 → 自举掩码断言；裁剪目标的四种情形 → 逐项梯度断言；A3C 的 $1/B$ 方差缩减 → 固定策略下的受控实验与幂律拟合。

三个最值得记住的实测结论：

- 三次多项式只给端点条件时峰值会超限（ $\Delta q = 1 \text{ rad}$ 、 $T = 1 \text{ s}$ 时峰值加速度 6 rad/s^2 ，是给定的 2 rad/s^2 上限的 3 倍）——限制必须显式写进规划器，不能靠“留余量”。
- 表格 Q-learning 稳定卡在 150 分附近，加密格子反而更差；DQN 直接吃连续观测可以打满 500 ——格子法的瓶颈是离散化误差，不是调参。
- PPO 的裁剪把每次更新的近似 KL 从 0.261 压到 0.01449（约 18 倍），而学习率、批大小、epoch 数都完全相同——“更稳定”是可以量化的，不需要停留在定性描述。